点云处理仿真系统教学材料

第一章：教学概述

1.1 教学目标

通过本系统的学习，学生将能够：

1. **理解激光雷达点云的生成原理**
2. 掌握光线投射和碰撞检测
3. 理解点云的空间分布特征
4. 了解真实传感器的噪声特性
5. **掌握点云下采样算法**
6. 学习体素网格下采样
7. 理解采样密度与精度的权衡
8. 掌握参数选择原则
9. **学习点云聚类算法**
10. 理解 DBSCAN 密度聚类原理
11. 掌握聚类参数的调优方法
12. 学习边界框计算
13. **了解点云处理流程**
14. 原始点云 → 下采样 → 聚类的完整流程
15. 各阶段的作用和参数影响

1.2 适用对象

- 自动驾驶技术学习者
- 机器人感知工程师
- 计算机视觉研究者
- 对点云处理感兴趣的学生

1.3 先修知识

知识领域	要求程度	说明
Python 编程	基础	能够阅读和理解 Python 代码
线性代数	基础	向量、矩阵运算
数据结构	基础	树结构、空间索引
概率统计	可选	理解噪声和统计特性

第二章：点云生成原理

2.1 激光雷达工作原理

LiDAR (Light Detection and Ranging) 通过发射激光脉冲并测量返回时间来确定距离：

```
距离 = (光速 × 飞行时间) / 2
```

机械旋转式 LiDAR

```
        ┌─────────────────┐
        │    旋转头部     │ ← 水平360°旋转
        │  ┌──────────┐  │
        │  │ 激光单元 │  │ ← 多线束垂直分布
        │  └──────────┘  │
        ├─────────────────┤
        │    电机传动     │
        └─────────────────┘
```

2.2 射线碰撞检测

本系统使用光线投射模拟激光雷达：

射线-AABB 相交（Slab 方法）

```
def ray_intersect_box(ray, box):
    """
    Slab 方法：将 AABB 看作三个无限 Slab 的交集

    对每个轴 (X, Y, Z)：
    1. 计算射线进入和离开该 Slab 的参数 t1, t2
    2. 更新全局的 tmin = max(tmin, t1)
    3. 更新全局的 tmax = min(tmax, t2)
    4. 如果 tmin > tmax，无交点
    """
    for i in range(3):
        t1 = (box.min[i] - ray.origin[i]) / ray.direction[i]
        t2 = (box.max[i] - ray.origin[i]) / ray.direction[i]
        if t1 > t2: t1, t2 = t2, t1
        tmin = max(tmin, t1)
        tmax = min(tmax, t2)
        if tmin > tmax: return False
    return True, tmin
```

射线-圆柱相交

```
def ray_intersect_cylinder(ray, cylinder):
    """
    圆柱方程: x² + z² = r²
    射线: P(t) = origin + t * direction

    代入得到二次方程:
    (dx² + dz²)t² + 2(ox·dx + oz·dz)t + (ox² + oz² − r²) = 0

    判别式 >= 0 时有交点
    """
    a = dx**2 + dz**2
    b = 2 * (ox*dx + oz*dz)
    c = ox**2 + oz**2 - r**2

    discriminant = b**2 - 4*a*c
    if discriminant < 0:
        return False, inf

    t = (-b - sqrt(discriminant)) / (2*a)
    # 检查高度限制
```

2.3 真实感噪声模拟

真实激光雷达存在多种误差源：

```
# 1. 距离相关的测量噪声
noise_std = base_noise + distance * noise_factor

# 2. 入射角影响检测概率
if cos_incidence < threshold:
    dropout_probability += 0.3

# 3. 随机 dropout（模拟反射率差异）
if random() < dropout_prob:
    continue  # 丢弃该点

# 4. 角度抖动（机械旋转不稳定性）
jitter = normal(0, noise_std * 0.5)
```

第三章：点云下采样

3.1 为什么需要下采样

原始点云通常非常密集（每秒数十万点），直接处理效率低。下采样可以： - 减少数据量，加速后续处理 - 去除冗余信息 - 均匀化点云密度

3.2 体素网格下采样

算法原理

```
1. 将空间划分为大小为 v 的立方体（体素）
2. 计算每个点所属的体素索引：(x/v, y/v, z/v)
3. 同一体素内的点取质心作为代表点
```

代码实现

```
def voxel_grid_downsample(points, voxel_size):
    """
    体素网格下采样

    参数:
        points: Nx3 点云数组
        voxel_size: 体素边长 (米)

    返回:
        降采样后的点云
    """
    # 计算边界
    min_bound = np.min(points, axis=0)

    # 计算体素索引
    voxel_indices = np.floor((points - min_bound) / voxel_size).astype(np

    # 创建唯一体素键
    shift = 20000  # 处理负数
    voxel_keys = ((voxel_indices[:, 0] + shift) * 40001 * 40001 +
                  (voxel_indices[:, 1] + shift) * 40001 +
                  (voxel_indices[:, 2] + shift))

    # 找出唯一体素
    unique_keys, inverse = np.unique(voxel_keys, return_inverse=True)

    # 计算每个体素的质心
    num_voxels = len(unique_keys)
    downsampled = np.zeros((num_voxels, 3))
    for i in range(3):
        downsampled[:, i] = (np.bincount(inverse, weights=points[:, i]) /
                             np.bincount(inverse))

    return downsampled
```

参数选择

体素大小	效果	适用场景
0.05m	轻微降采样，细节保留好	高精度需求
0.1-0.2m	中等降采样，平衡精度和效率	通用场景
0.5m+	大幅降采样，只保留主要结构	快速预览

3.3 随机下采样

```
def random_downsample(points, target_count):
    """
    随机均匀采样

    优点: 实现简单，速度极快
    缺点: 可能丢失重要特征点
    """
    indices = np.random.choice(len(points), target_count, replace=False)
    return points[indices]
```

第四章：点云聚类

4.1 聚类的目的

将点云分割成有意义的簇（如不同的物体），是目标检测和跟踪的基础。

4.2 DBSCAN 算法

算法原理

DBSCAN (Density-Based Spatial Clustering) 是基于密度的聚类算法：

```
核心概念：
- ε-邻域：以点 p 为中心，半径 ε 的球内所有点
- 核心点：ε-邻域内至少有 min_samples 个点
- 边界点：不是核心点，但在某核心点的邻域内
- 噪声点：既不是核心点也不是边界点

聚类过程：
1. 找出所有核心点
2. 从未访问的核心点开始，创建新簇
3. 将该点的 ε-邻域内的所有核心点加入簇
4. 递归扩展，直到没有新的核心点可加入
5. 重复直到所有核心点都被访问
```

代码实现

```
from scipy.spatial import cKDTree

def dbscan_clustering(points, eps, min_samples):
    """
    DBSCAN 聚类（KDTree 加速版）

    参数:
        points: Nx3 点云数组
        eps: 邻域半径（米）
        min_samples: 核心点最小邻居数

    返回:
        labels: 聚类标签数组，-1 表示噪声
    """
    num_points = len(points)
    labels = np.full(num_points, -2, dtype=np.int32)  # -2: 未访问

    # 使用 KDTree 加速邻域搜索
    tree = cKDTree(points)
    neighbors_list = tree.query_ball_point(points, eps)

    cluster_id = 0

    for i in range(num_points):
        if labels[i] != -2:
            continue

        neighbors = neighbors_list[i]

        if len(neighbors) < min_samples:
            labels[i] = -1  # 噪声
            continue

        # 开始新聚类
        labels[i] = cluster_id
        seed_set = list(neighbors)
```

```
        seed_set.remove(i)

        j = 0
        while j < len(seed_set):
            q = seed_set[j]

            if labels[q] == -1:
                labels[q] = cluster_id  # 边界点
            elif labels[q] == -2:
                labels[q] = cluster_id

                q_neighbors = neighbors_list[q]
                if len(q_neighbors) >= min_samples:
                    # q 是核心点，扩展
                    for n in q_neighbors:
                        if n not in seed_set:
                            seed_set.append(n)

            j += 1

        cluster_id += 1

    return labels
```

参数选择

参数	影响	选择建议
eps	邻域半径	使用 k-distance 图找拐点
min_samples	最小邻居数	通常为 2×维度 = 6

k-distance 图方法：

```
# 计算每个点到第 k 近邻的距离
distances, _ = tree.query(points, k=min_samples)
k_distances = distances[:, -1]
k_distances.sort()

# 绘制并找拐点
plt.plot(k_distances)
plt.ylabel(f'{min_samples}-NN distance')
```

4.3 边界框计算

```
def compute_bounding_boxes(points, labels):
    """
    为每个聚类计算轴对齐包围盒 (AABB)

    返回每个簇的:
    - min_bound: 最小角点
    - max_bound: 最大角点
    - center: 中心点
    - size: 尺寸
    """
    bounding_boxes = []
    unique_labels = np.unique(labels[labels >= 0])

    for cluster_id in unique_labels:
        cluster_points = points[labels == cluster_id]

        min_bound = np.min(cluster_points, axis=0)
        max_bound = np.max(cluster_points, axis=0)

        bounding_boxes.append({
            'cluster_id': cluster_id,
            'min_bound': min_bound,
            'max_bound': max_bound,
            'center': (min_bound + max_bound) / 2,
            'size': max_bound - min_bound,
            'num_points': len(cluster_points)
        })

    return bounding_boxes
```

第五章：系统架构

5.1 模块划分

```
┌──────────────────────────────────────────────────────────────┐
│                          MainWindow                          │
│  ┌──────────────────────────────┐  ┌──────────────────────┐  │
│  │          GLWidget            │  │   Control Panels     │  │
│  │  ┌────────────────────────┐  │  │  ┌────────────────┐  │  │
│  │  │         Scene          │  │  │  │ Scan Params    │  │  │
│  │  │  ┌────┐  ┌──────────┐  │  │  │  │                │  │  │
│  │  │  │Cam │  │Pipeline  │  │  │  │  │                │  │  │
│  │  │  │    │  │Result    │  │  │  │  │ Pipeline Panel │  │  │
│  │  │  ┌────────────────┐ │  │  │  │  │                │  │  │
│  │  │  │   Environment  │ │  │  │  │  │                │  │  │
│  │  │  │                │ │  │  │  │  │                │  │  │
│  │  │  └────────────────┘ │  │  │  │  │                │  │  │
│  │  └────────────────────┘  │  │  │  │                │  │  │
│  │                          │  │  │  │                │  │  │
│  │                          │  │  │  │                │  │  │
└──────────────────────────────────────────────────────────────┘
```

5.2 数据流

```
环境物体 ──► 射线投射 ──► 原始点云
                              │
                              ▼
                          体素下采样
                              │
                              ▼
                          DBSCAN聚类
                              │
                              ▼
                        边界框 + 可视化
```

第六章：实验指导

6.1 环境配置

```
# 创建虚拟环境
python -m venv venv
source venv/bin/activate

# 安装依赖
pip install PyQt5 PyOpenGL numpy scipy
```

6.2 运行程序

```
python main.py
```

6.3 实验内容

实验一：下采样效果对比

体素大小	原始点数	下采样后	保留比例
0.1m	~5760	~1200	21%
0.2m	~5760	~400	7%
0.5m	~5760	~100	2%

观察要点： - 体素越大，点数越少 - 过大的体素会丢失物体细节

实验二：聚类参数调优

eps	min_samples	簇数	噪声点
0.3	5	15+	多
0.5	5	7-8	少
1.0	5	3-4	很少

观察要点： - eps 太小：过度分割，噪声多 - eps 太大：不同物体被合并 - min_samples 影响噪声判定

实验三：多阶段流程

1. 观察原始点云（按物体类型着色）
2. 切换到下采样阶段，观察点数减少
3. 切换到聚类阶段，观察：
4. 不同物体被分到不同簇
5. 噪声点显示为灰色
6. 边界框包围每个簇

6.4 思考题

1. **下采样**
2. 体素网格下采样与随机下采样各有什么优缺点?
3. 如何选择合适的体素大小?
4. **聚类**
5. DBSCAN 与 K-Means 有什么区别?
6. 为什么 DBSCAN 能识别噪声点?
7. 如何确定最优的 eps 和 min_samples?
8. **应用**
9. 聚类结果如何用于目标检测?

10. 边界框信息如何帮助路径规划?

第七章：关键代码示例

7.1 完整处理流程

```
# 1. 生成原始点云
scene.generate_point_cloud()
raw_points = scene.get_raw_point_cloud()

# 2. 创建处理流水线
pipeline = ProcessingPipeline()
pipeline.set_raw_point_cloud(raw_points)

# 3. 下采样
pipeline.set_voxel_size(0.2)
pipeline.process_downsampling()
downsampled = pipeline.downsampled_result.point_cloud

# 4. 聚类
pipeline.set_eps(0.5)
pipeline.set_min_samples(5)
pipeline.process_clustering()
clustered = pipeline.clustered_result

# 5. 获取结果
labels = clustered.cluster_labels
bounding_boxes = clustered.bounding_boxes
print(f"发现 {len(bounding_boxes)} 个簇")
```

7.2 可视化着色

```
def draw_clustered_point_cloud(points, labels):
    """绘制聚类后的点云"""
    colors = get_cluster_colors(num_clusters)

    glBegin(GL_POINTS)
    for i, (point, label) in enumerate(zip(points, labels)):
        if label < 0:
            glColor4f(0.5, 0.5, 0.5, 0.7)  # 噪声 - 灰色
        else:
            color = colors[label]
            glColor4f(color[0], color[1], color[2], 0.9)

        glVertex3f(point[0], point[1], point[2])
    glEnd()
```

附录：参考资料

A. 推荐阅读

1. **点云处理**
2. 《Point Cloud Library》文档
3. Open3D 官方教程
4. **聚类算法**
5. 《Pattern Classification》- Duda
6. DBSCAN 原论文: Ester et al., 1996
7. **激光雷达**

8. Velodyne LiDAR 官方文档
9. 《Self-Driving Cars》- Coursera

B. 在线资源

- Open3D: http://www.open3d.org/
- PCL: https://pointclouds.org/
- scikit-learn DBSCAN: https://scikit-learn.org/stable/modules/clustering.html#dbscan

---

附录：快捷键参考

快捷键	功能
`R`	重置视角
`1`	俯视图
`2`	正视图
`3`	侧视图
`4`	透视图
`5`	等轴视图
`ESC`	退出程序
鼠标左键拖拽	旋转视角
鼠标滚轮	缩放

本教材配套项目：点云处理仿真系统

最后更新：2026年2月